

1 資料結構

1.1 DSU

```
struct DSU {
    vector<int> f, sz;
    int cnt;
    DSU(int n) : f(n), sz(n) {
        for (int i = 0; i < n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {
        if (x == f[x]) return x;
        return f[x] = find(f[x]);
    }
    void merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return;
        if (sz[x] < sz[y])
            swap(x, y);
        f[y] = x;
        sz[x] += sz[y];
        cnt--;
    }
    bool same(int a, int b) {
        return find(a) == find(b);
    }
    int groups() { // 找有幾群
        return cnt;
    }
};
```

1.2 BIT

```
struct Binary_Indexed_Tree{
    int n;
    vector<long long> bit;
    int lowbit(int x){
        return x&-x;
    }
    void init(int _n){
        n = _n+1;
        bit = vector<long long>(n,0);
    }
    void update(int x,int v){
        for(; x<n; x+=lowbit(x)){
            bit[x] += v;
        }
    }
    long long query(int x){
        long long ret = 0;
        for(; x>0; x-=lowbit(x)){
            ret += bit[x];
        }
        return ret;
    }
} BIT;
```

1.3 SEG

```
#define cl(x) (x<<1)
#define cr(x) (x<<1)+1
const int N;
int seg[4*N];
void pull(int id){
    seg[id]=max(seg[cl(id)],seg[cr(id)])
}
void build(int id,int l,int r){
    if(l==r){
        seg[id]=arr[l];
        return ;
    }
    int mid=(l+r)>>1;
    build(cl(id),l,mid);
```

```
    build(cr(id),mid+1,r);
    pull(id);
}
void update(int id,int l,int r,int x,int v){
    if(l==r){
        seg[id]=v;
        return ;
    }
    int mid=(l+r)>>1;
    if(x<=mid){
        update(cl(id),l,mid,x,v);
    }
    if(mid<x){
        update(cr(id),mid+1,r,x,v);
    }
    pull(id);
}
int query(int id,int l,int r,int sl,int sr){
    if(sl<=l&&r<=sr){ //目前這個區間在查詢區間內
        return seg[id];
    }
    int mid=(l+r)>>1,res=0;
    if(sl<=mid){ //左區間跟查詢區間有交集
        res=max(res,query(cl(id),l,mid,sl,sr));
    }
    if(mid<sr){ //右區間跟查詢區間有交集
        res=max(res,query(cr(id),mid+1,r,sl,sr));
    }
    return res;
}
struct node{
    int sum,pre,suf,ans; //總和、最大前綴、最大後綴、
    //當前區間答案
}seg[4*N];
void pull(int id){
    int l=cl(id),r=cr(id); //左右兩個子區間的index
    seg[id].sum=seg[l].sum+seg[r].sum;
    seg[id].pre=max(seg[l].pre,seg[l].sum+seg[r].pre);
    ;
    seg[id].suf=max(seg[r].suf,seg[r].sum+seg[l].suf);
    ;
    seg[id].ans=max({seg[l].ans,seg[r].ans,seg[l].suf
        +seg[r].pre});
}
```

2 最短路

2.1 Dijkstra

```
#include <iostream>
#include <vector>
#include <utility>
#include <queue>
using namespace std;

#define pii pair<int,int>
#define INF 1e18
#define int long long

vector<pii> vec[100005];
vector<int> vis(100005);

int n, m;

void dijkstra(int s,int t){ //起點·終點
    int dis[100005];
    for(int i=0;i<100005;i++){ //初始化
        dis[i]=INF; //值要設為比可能的最短路徑權重還要
        //大的值
    }
    dis[s]=0;
    priority_queue<pii,vector<pii>,greater<pii>> pq;
    //以小到排序
    pq.push({dis[s],s});
    while(pq.empty()==0){
        int u=pq.top().second;
        pq.pop();
        if (vis[u]) continue;
```

```

        vis[u]=1;

        for (int i = 0; i < vec[u].size(); i++) {
            int v = vec[u][i].first;
            int w = vec[u][i].second;
            if(dis[u]+w < dis[v]){//鬆弛
                dis[v] = dis[u]+w;
                pq.push({dis[v],v});
            }
        }
    }

    for (int i = 1; i <= n; i++) {
        cout << dis[i] << " ";
    }
}

main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b, c;
        cin >> a >> b >> c;
        vec[a].push_back({b, c});
    }

    dijkstra(1, -1);
}

```

2.2 SPTree

```

#include <iostream>
#include <vector>
#include <utility>
#include <queue>
#include <tuple>
using namespace std;

#define pii pair<int,int>
#define INF 1e18
#define int long long

vector<tuple<int, int, int>> vec[200005];
vector<int> vis(200005);
vector<int> path(200005);

int n, m;

void dijkstra(int s,int t){//起點・終點
    int dis[200005];
    for(int i=0;i<200005;i++){//初始化
        dis[i]=INF;//值要設為比可能的最短路徑權重還要
        大的值
    }
    dis[s]=0;
    priority_queue<pii,vector<pii>,greater<pii>> pq;
    //以小到大排序
    pq.push({dis[s],s});
    while(pq.empty()==0){
        int u=pq.top().second;
        pq.pop();
        if (vis[u]) continue;
        vis[u]=1;

        for (auto [v, w, i] : vec[u]) {
            if(dis[u]+w < dis[v]){//鬆弛
                dis[v] = dis[u]+w;
                pq.push({dis[v],v});
                path[v] = i;
            }
        }
    }

    for (int i = 2; i <= n; i++) {
        cout << path[i] << " ";
    }
}

main() {
    cin >> n >> m;

```

```

        for (int i = 1; i <= m; i++) {
            int a, b, c;
            cin >> a >> b >> c;
            vec[a].push_back({b, c, i});
            vec[b].push_back({a, c, i});
        }

        dijkstra(1, -1);
    }
}

```

2.3 Bellman

```

#include <iostream>
#include <vector>
#include <utility>
#include <queue>
#include <tuple>
#include <climits>
using namespace std;

#define int long long

vector<tuple<int, int, int>> edge[2005];
int n, m;

void bellman(int s, int t) {
    int dis[2005];
    for (int i = 0; i < 2005; i++) dis[i] = LLONG_MIN;
    dis[s] = 0;

    for (int iter = 1; iter <= n; iter++) {
        vector<int> chang;
        for (int u = 1; u <= n; u++) {
            if (dis[u] == LLONG_MIN) continue;
            for (auto [from, v, w] : edge[u]) {
                if (dis[u] + w < dis[v]) {
                    dis[v] = dis[u] + w;
                    if (iter == n) chang.push_back(v);
                }
            }
        }

        if (iter == n) {
            queue<int> q;
            vector<int> vis(2005);
            for (auto i : chang) q.push(i);

            while (!q.empty()) {
                int u = q.front(); q.pop();
                if (u == n) {
                    cout << "inf";
                    exit(0);
                }
                for (auto [u, v, w] : edge[u]) {
                    if (!vis[v]) {
                        vis[v] = true;
                        q.push(v);
                    }
                }
            }
        }

        cout << dis[n];
    }
}

signed main() {
    cin >> n >> m;
    for (int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edge[u].push_back({u, v, w});
    }
    bellman(1, -1);
}

```

2.4 Floyd-Warshall

```
for(int k=1;k<=N;k++){//窮舉中繼點k
    for(int i=1;i<=N;i++){
        for(int j=1;j<=N;j++){//窮舉點對(i,j)
            dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j]);
        }
    }
}
```

3 圖論

3.1 bfs

```
bool vis[MXN];
void bfs(int s){
    queue<int> q;
    q.push(s);
    vis[s]=1;
    while(!q.empty()){
        int x=q.front();q.pop();
        for(int i:edge[x]){
            if(!vis[i])
                q.push(i),vis[i]=1;
        }
    }
}
```

3.2 dfs

```
void dfs(int x){
    vis[x]=1;
    for(int i:edge[x]){
        if(!vis[i])
            dfs(i);
    }
}
```

3.3 ola

```
//歐拉路徑(無向圖)兩個奇數點或者全偶點
//歐拉路徑(有向圖)條件:起點out=in+1,終點in=out+1,其他
//點in=out
//歐拉迴路(無向圖)所有點偶數點
//歐拉迴路(有向圖)所有點in=out
vector<int> anse;//邊
vector<int> ansp;//點
vector<bool> vis(m,false);
function<void(int)> dfs=[&](int u){
    while(!mp[u].empty()){
        auto [x,e]=mp[u].back();
        mp[u].pop_back();
        if(vis[e]){
            continue;
        }
        vis[e]=true;
        dfs(x);
        anse.pb(e);
    }
    ansp.pb(u);
};
```

3.4 topo

```
vector<int> edge[MAXN], ans;
int deg[MAXN];

void topo(int n) {
    queue<int> q;

    for (int i = 1; i <= n; i++)
```

```
        if (!deg[i])
            q.push(i);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);

        for (int v : edge[u]) {
            deg[v]--;
            if (!deg[v])
                q.push(v);
        }
    }
}
```

4 數學

4.1 Fast Power

```
int ksm(int x, int y) {
    int ans = 1;
    while (y) {
        if (y & 1)
            ans = ans * x;
        x = x * x;
        y >>= 1;
    }
    return ans;
}

// 矩陣乘法
vector<vector<int>> mul(const vector<vector<int>> &A,
    const vector<vector<int>> &B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) if (A[i][k]) {
            for (int j = 0; j < n; j++) {
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
            }
        }
    }
    return C;
}

// 矩陣快速幂
vector<vector<int>> ksm(vector<vector<int>> base, int exp) {
    int n = base.size();
    vector<vector<int>> res(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (exp > 0) {
        if (exp & 1) res = mul(res, base);
        base = mul(base, base);
        exp >>= 1;
    }
    return res;
}
```

4.2 GCD

```
int gcd(int a, int b) {
    int r = a % b;
    while (r != 0) {
        a = b, b = r, r = a % b;
    }
    return b;
}
```

4.3 EXGCD

```
#define ll long long
int extgcd(int a, int b, int &x, int &y) {
    if(b == 0){
        x = 1;
        y = 0;
        return a;
    }
    int GCD=extgcd(b, a%b, y, x);
    y -= a / b * x;
    return GCD;
}

int main(){
    int a, b, x, y;
    cin >> a >> b;
    int GCD = extgcd(a, b, x, y);
}
```

4.4 C n 取 k

$$\frac{n!}{k!(n-k)!} \equiv n! \cdot (k!)^{-1} \cdot ((n-k)!)^{-1} \pmod{p} \quad (1)$$
$$\equiv n! \cdot (k!)^{p-2} \cdot ((n-k)!)^{p-2} \pmod{p}$$

```
#define MXN 1'000'005
#define N 1'000'000
#define ll long long
ll fac[MXN], inv[MXN];
fac[0] = 1; // 0! = 1
for(int i = 1; i <= N; i++)    fac[i] = fac[i-1] * i
    % mod;
inv[N] = fpow(fac[N], mod-2); // 快速幂
for(int i = N-1; i >= 0; i--)    inv[i] = inv[i+1] * (i
    +1) % mod;
```

4.5 錯排

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

$$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$$

4.6 離散化

```
//vector<int> arr;
vector<int> tmp(arr); //將arr複製到tmp
sort(tmp.begin(), tmp.end());
tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
for (int i = 0; i < n; i++)
    arr[i] = lower_bound(tmp.begin(), tmp.end(), arr[i]
        ]) - tmp.begin();
```

4.7 快速乘

```
#define ll long long
const ll MOD = 1e9 + 7; //
1000000007 是個質數
ll mypow(ll x, ll y){
    ll ans = 1;
    while(y){
        if(y & 1) ans = (ans * x) % MOD; // 這裡 %
        MOD
        x = (x * x) % MOD; // 這裡 %
        MOD
        y >>= 1;
    }
    return ans;
}
```

4.8 容斥

```
for (int s = 1; s < 1<<n; s++) {
    int sign = -1;
    if (__builtin_popcount(s)&1) sign = 1;
    int mul = 1;
    for (int j = 0; j < n; j++) {
        if (s & (1<<j)) {
            mul = mul * a[j];
        }
    }
    ans += sign * (n / mul);
}
```

5 字串

5.1 KMP

```
vector<int> build_lps(string s){
    int n = s.size();
    vector<int> lps(n);
    for(int i=1,j=0;i<n;i++){
        while(j>0 && s[i]!=s[j]) j=lps[j-1];
        if(s[i]==s[j]) j++;
        lps[i]=j;
    }
    return lps;
}
```

6 DP

6.1 0-1 背包

```
for(int i = 1; i <= n;++i){
    for(int w = W; w >= 0;--w)
        dp[w] = max(w - c[i] >= 0 ? dp[w - c[i]] + v[i] :
            0,dp[w]);}
```

6.2 無限背包

```
for(int w = 0; w <= W; ++w){
    for(int i = 1; i <= n; ++i){
        dp[w] = max(dp[w],\
            w-c[i] >= 0 ? dp[w-c[i]] + v[i] : 0);
    }
}
```

6.3 二進制分組

```
//目前物品最多拿 k 個
int s = 0;
vector<pair<int,int>> A; // A 是儲存所有捆
for(int r = 1; r * 2 <= k; r *= 2){
    s += r;
    A.push_back({r*c[i],r*v[i]}); // r 個一捆
}
A.push_back({(k-s)*c[i],(k-s)*v[i]}); // 最後一捆
```

6.4 位元背包

```
for(int i = 0; i < n; ++i) dp[(1<<i)][i] = true; // 初始化 (自己能走到自己)
for(int s = 1; s < (1<<n); ++s){
    for(int j = 0; j < n; ++j){
        if(s & (1<<j)) continue;
        // s 還不能經過 j · 因為一個點只能經過一次
        for(int i = 0; i < n; ++i){ // 窮舉 s 內的終點
            if(!(s & (1<<i))) continue; // s 必須得經過 i
            if(edge[i][j]){ // 有 i -> j 的邊
                dp[s | (1<<j)][j] |= dp[s][i];
            }
        }
    }
}
bool ans = false;
for(int i = 0; i < n; ++i) ans |= dp[(1<<n)-1][i];
```

6.5 LCS

```
for(int i = 0; i <= n; ++i) dp[i][0] = 0;
for(int j = 0; j <= m; ++j) dp[0][j] = 0;

for(int i = 1; i <= n; ++i){
    for(int j = 1; j <= m; ++j){
        dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        if(s[i] == t[j]) dp[i][j] = max(dp[i][j], dp[i-1][j-1]+1);
    }
}

cout << dp[n][m];
```

6.6 LIS

```
#define all(v) (v).begin(), (v).end()

vector<int> dp;
for(int i = 0; i < n; ++i){
    auto it = lower_bound(all(dp), A[i]);
    if(it == dp.end()){
        dp.push_back(A[i]);
    }else{
        *it = A[i];
    }
}
cout << dp.size() << '\n';
```

6.7 SOS

```
for(int i = 0; i < (1<<N); ++i) F[i] = A[i]; // 預處理狀態權重

for(int i = 0; i < N; ++i)
    for(int s = 0; s < (1<<N); ++s)
```

```
if (s & (1 << i))
    F[s] += F[s ^ (1 << i)];
```

//窮舉子集合

```
for(int s = mask; s ; s = (s-1)&mask;)
```

6.8 編輯距離

```
dp[0][0] = 0; //Edge Case, 兩個空字串不需要操作
for(int i = 1; i <= n; ++i) dp[i][0] = i; //如果一個字串為空 則最小成本就是刪掉另一個字串
for(int j = 1; j <= m; ++j) dp[0][j] = j;

//老套的從小大到轉移
for(int i = 1; i <= n; ++i)
    for(int j = 1; j <= m; ++j){
        if(s[i] == t[j]) dp[i][j] = dp[i-1][j-1]; //結尾相同 · 所以不用花成本
        dp[i][j] = min({
            dp[i][j],
            dp[i-1][j] + 1, //刪除 s[i] 花費成本 1
            dp[i][j-1] + 1, //刪除 t[j] 花費成本 1
            dp[i-1][j] + 1, //插入一個字元在 t[j] 後和 s[i] 配對 花費成本 1
            dp[i][j-1] + 1, //插入一個字元在 s[i] 後和 t[j] 配對 花費成本 1
            dp[i-1][j-1] + 1 //替換 t[i] 換成跟 s[j] 一樣的花費成本 1
        });
    }
cout << dp[n][m];
```

6.9 子集和

```
bs dp;
dp[0] = 1; // 0 就是不取東西就能湊出來
for(int i = 1; i <= n; ++i){
    dp = dp | (dp<<A[i]);
}
cout << dp[K];
```

7 雜項

7.1 Cool Things

```
priority_queue<T> //由大到小
priority_queue<T, vector<T>, greater<T>> //由小到大
__builtin_popcountll // 二進位有幾個1(記得這是long long)
__builtin_clzll // 左起第一個1之前0的個數
__builtin_parityll // 1的個數的奇偶性
__builtin_mul_overflow(a,b,&h) // a*b是否溢位, h = a * b;
__builtin_add_overflow(a,b,&h)
```